RESOURCES & FETCHING

I. Lesson

What is a Resource?

A **Resource** in Leptos is a reactive wrapper around an **async task**. You give it a ***source*** (what the task depends on) and a ***fetcher*** (an async function). It watches the source; whenever it changes, the fetcher runs again.

II. Exercice

CSR -> reqwasm SSR -> reqwest

Feature	`reqwasm`	`reqwest`
Purpose	HTTP client for **WebAssembly (browser)**	General-purpose HTTP client for **native/server** Rust
Environment	**Browser-only** (WASM)	**Native & server-side** (tokio/async-std)
Backend	Uses the browser's `fetch` API	Built on **Hyper** and native I/O
Best use cases	Frontend Rust apps (Leptos, Yew) running in the browser	Backend services, CLIs, servers, SSR code
Feature set	Simple GET/POST, headers, JSON via `fetch`	Full-featured: redirects, cookies, proxies, HTTP/2, gzip, streaming, timeouts, multipart, TLS
CORS behavior	Must obey **browser CORS** rules	Controlled by your server/client config (no browser CORS limits)
Limitations	No raw sockets; limited streaming; no native TLS; no filesystem/proxy access	Heavier dependency footprint; not available in browser WASM
Leptos fit	Use in **WASM client** code	Use on the **server side** (SSR, server functions, APIs)

```toml
[package]
name = "server"
version = "0.1.0"
edition = "2024"

[dependencies]
app = { path = "../app", default-features = false, features = ["ssr"] }
leptos = { workspace = true, features = [ "ssr" ]}
leptos_axum.workspace = true

axum.workspace = true
tokio.workspace = true
tower.workspace = true
tower-http.workspace = true
log.workspace = true
```

Resource vs LocalResource

🔎 Resource vs LocalResource in Leptos

Feature	`Resource`	`LocalResource`
Scope	App-wide / can be shared	Component-local
Server-Side Support	Works on **client and server** (SSR/CSR)	**Client-only** (no SSR)
Sharing	Can be **shared across components**	**Isolated** to the creating component
Caching	Often **cached during SSR** (rehydrates on client)	**No caching** by default
Use Case	Reusable data (e.g., user profile, config)	Component-specific fetch (e.g., dropdown options)
Lifetime	Tied to the **reactive scope** it's created in	Scope-tied, typically for small **UI widgets**

```
<Suspense fallback=move || {
    view! {
        <div>
            <p>"Loading..."</p>
        </div>
    }
}>
```

```rust
#[server(GetPosts)]
pub async fn get_posts() -> Result<Vec<PostDto>, ServerFnError> {
    let resp = match reqwest::get(format!("{BASE_URL}?_limit={LIMIT}")).await {
        Ok(response) => response,
        Err(err) => return Err(ServerFnError::ServerError(err.to_string())),
    };

    let posts = match resp.json::<Vec<PostDto>>().await {
        Ok(data) => data,
        Err(err) => return Err(ServerFnError::ServerError(err.to_string())),
    };

    Ok(posts)
}
```

Let’s code!

Your turn!